

Redes de Datos

Boletín de Ejercicios Temas 2 y 3

Francisco Javier Mercader Martínez

- 1) ¿Por qué el máximo tiempo de vida de un paquete T , tiene que ser suficientemente grande para asegurar que no solo el paquete, pero también los asentimientos, se han desvanecido?

Se espera un tiempo largo, equivalente al doble del tiempo de vida de un segmento (60 segundos), después de enviar todos los segmentos antes de completar el cierre por las siguientes razones:

- **Para garantizar un cierre ordenado:** Es posible que existan mensajes de asentimiento (ACK) perdidos en la red. Si esto sucede, este tiempo de espera permite que se reenvíe el mensaje FIN necesario para cerrar la comunicación correctamente.
 - **Para evitar cruces de datos:** Si no se espera a que estos paquetes y sus correspondientes asentimientos desaparezcan de la red, el tráfico residual podría inferir de manera negativa con una conexión subsiguiente.
- 2) ¿Qué ocurre si el usuario de una entidad de transporte envía un mensaje de longitud cero? Reflexiona sobre el significado de la respuesta.
- 3) Imagina que se utilizara un *handshake* bidireccional en lugar de uno tridireccional para establecer las conexiones. En otras palabras, el tercer mensaje no es necesario. ¿Son posibles ahora los bloqueos? Por un ejemplo o demuestra que no existe ninguno.

El *Three-Way Handshake* se utiliza en TCP específicamente **”por robustez frente a duplicados retardados”**.

Para demostrar por qué el tercer mensaje es estrictamente necesario, el contenido plantea el siguiente escenario o ejemplo:

- Supongamos que en la red circulan **”copias retrasadas y duplicadas de SYN”** y estas acaban llegando a la parte pasiva (el servidor).
- Gracias a que existe un tercer mensaje de confirmación, si esto llega a pasar, **”ambos lados rechazarían el intento de conexión de forma limpia”**.

¿Qué ocurriría en un modelo bidireccional (sin el tercer mensaje)?

Si eliminamos el tercer paso, perderíamos esta robustez. El ejemplo se desarrollaría de la siguiente manera:

- 1) El servidor recibe un mensaje SYN que en realidad es un duplicado retrasado de una conexión anterior.
- 2) El servidor responde con un mensaje (SYN+ACK) y, al ser un modelo bidireccional, consideraría que la conexión ya está totalmente establecida por su parte.
- 3) El cliente, que no ha solicitado esa conexión recientemente, ignoraría el mensaje o enviaría un rechazo, pero el servidor ya habría comprometido sus recursos asumiendo que el cliente iba a empezar a enviar datos.

Por lo tanto, se demuestra que sin el tercer mensaje el sistema es incapaz de gestionar los **duplicados retardados**, dejando a la parte pasiva (servidor) en un estado inconsistente o colgado esperando una conexión fantasma.

- 4) Algunas otras políticas de equidad en el control de la congestión son Aumento Aditivo Disminución Aditiva (AIAD), Aumento Multiplicativo Disminución Aditiva (MIAD) y Aumento Multiplicativo Disminución Multiplicativa (MIMD). Analice estas tres políticas en términos de convergencia y estabilidad.

- **Falta de convergencia adecuada:** A diferencia de la política AIMD (Incremento Aditivo, Decremento Multiplicativo), las leyes de control para la congestión basadas en **MIAD, MIMD y AIAD no cumplen con los criterios de convergencias** necesarios.
- **Contraste con AIMD:** El modelo AIMD sí converge a una asignación que es justa y eficiente cuando todos los equipos ejecutan el algoritmo, utilizando este hecho para destacar por qué las otras tres alternativas (AIAD, MIAD y MIMD) se descartan.

Por lo tanto, la conclusión final sobre estas tres políticas es que fracasan en cumplir la convergencia hacia una asignación justa y eficiente en la red.

5) ¿Por qué existe UDP? ¿No habría bastado con dejar que los procesos de usuario enviaran paquetes IP sin procesar?

Podríamos definir la utilización de UDP como "casi equivalente a enviar paquetes". Sin embargo, no bastaría con enviar paquetes IP sin procesar porque UDP añade mecanismos mínimos pero esenciales para que los datos lleguen a la aplicación correcta.

UDP existe por las siguientes razones fundamentales:

- **Identificador de los procesos (Puertos):** Mientras que los paquetes IP llegan a la máquina, la cabecera UDP utiliza puertos para identificar los procesos de las aplicaciones específicas que transmiten y reciben los datos.
- **Buzones locales:** Estos puertos son números enteros de 16 bits que representan "buzones locales" que son cogidos prestados por los procesos. Un proceso de una aplicación se identifica por la tupla: dirección IP, protocolo y puerto. Sin UDP, los paquetes IP crudos no sabrían a qué aplicación entregar la información dentro del ordenador.
- **Integridad de los datos:** La cabecera UDP añade un *checksum* (suma de comprobación) de 16 bits para aportar fiabilidad a los datos transmitidos.

En resumen, UDP es el envoltorio mínimo necesario en la capa de transporte para aplicaciones que envían mensajes y no necesitan fiabilidad compleja o flujos de datos (como la Voz sobre IP, DNS o DHCP). Si la aplicación requiere mayor fiabilidad sobre UDP, tendrá que encargarse de conseguirla por sí misma.

6) Considere un protocolo simple a nivel de aplicación construido sobre UDP que permite a un cliente recuperar un archivo de un servidor remoto que reside en una dirección bien conocida. El cliente envía primero una solicitud con un nombre de archivo, y el servidor responde con una secuencia de paquetes de datos que contienen diferentes partes del archivo solicitado. Para garantizar la fiabilidad y la entrega secuencial, el cliente y el servidor utilizan un protocolo de parada y espera. Ignorando el evidente problema de rendimiento, ¿ves algún problema en este protocolo? Piensa detenidamente en la posibilidad de que los procesos se bloqueen.

El uso de un protocolo simple de "parada y espera" sobre UDP presenta varios problemas de diseño que pueden conducir a bloqueos (*deadlocks*). UDP no es fiable, por lo que la aplicación debe encargarse por completo de conseguir la fiabilidad.

Analizando los mecanismos que le faltan a este protocolo en comparación con un protocolo robusto como TCP, podemos identificar los siguientes problemas de bloqueo:

- **Pérdida de la solicitud inicial (Falta de temporizador en el receptor):** El algoritmo de la ventana deslizante y la parada y espera dependen del uso de un temporizador en el transmisor para detectar pérdidas y retransmitir. Sin embargo, si la solicitud inicial enviada por el cliente se pierde en la red y el cliente no implementa un temporizador para volver a solicitarla, el cliente se quedará bloqueado indefinidamente esperando el primer paquete de datos del servidor.
- **Problemas en el cierre y el último ACK:** En la teoría se destaca que el problema clave al liberar una conexión es "proporcionar fiabilidad mientras se produce la liberación". Cuando el servidor envía la última parte del archivo, el cliente responderá con un ACK y dará la descarga por finalizada. Si este último ACK se pierde por el camino, el servidor (que sigue en parada y espera) retransmitirá el último paquete indefinidamente al no recibir

confirmación. Dado que el cliente ya cerró la comunicación, el servidor se quedaría bloqueado. TCP soluciona este problema con un cierre ordenado simétrico y el estado de espera `TIME_WAIT`.

- **Vulnerabilidad a duplicados retardados (Ausencia de *Three-Way Handshake*):** Al no haber un mecanismo previo de señalización para establecer el estado de los extremos antes de la transferencia, el protocolo está expuesto. El establecimiento de conexión en tres pasos se hace "por robustez frente a duplicados retardados". Si una solicitud del cliente se retrasa mucho en la red y llega al servidor mucho después (cuando el cliente ya ha abandonado o reiniciado la transferencia), el servidor empezará a enviar un archivo a un cliente que no lo espera, desperdiciando ancho de banda y bloqueando recursos del servidor de forma innecesaria.

7) El cliente envía una petición de 128 bytes a un servidor situado a 100 km de distancia a través de una fibra óptica de 1 gigabit. ¿Cuál es la eficiencia de la línea durante la llamada a procedimiento remoto?

Datos del problema

- **Capacidad de la línea (R):** $1 \text{ Gbps} = 10^9 \text{ bps}$
- **Tamaño del paquete:** $128 \text{ Bytes} = 1024 \text{ bits}$
- **Distancia:** 100 km
- **Velocidad en fibra:** $2 \times 10^5 \text{ km/s}$

Pasos del cálculo

1) Cálculo del retardo de propagación (D):

$$D = \frac{100 \text{ km}}{2 \times 10^5 \text{ km/s}} = 0.0005 \text{ s} = 0.5 \text{ ms}$$

2) Cálculo del tiempo de ida y vuelta (RTT):

El RTT se define matemáticamente como $2D$.

$$RTT = 2 \times 0.5 \text{ ms} = 1 \text{ ms}$$

3) Tasa de transmisión de paquetes:

Aquí solo podemos enviar una solicitud y recibir una respuesta cada milisegundo ya que la tasa de paquetes por segundo está limitada por el tiempo que tarda la señal en ir y volver ($1/2D$):

$$\text{Tasa} = \frac{1 \text{ paquete}}{1 \text{ ms}} = 1000 \text{ paquetes/s}$$

4) Caudal útil (Throughput):

Multiplicamos la cantidad de paquetes por segundo por el tamaño del paquete en bits para saber cuánto tráfico real estamos enviando al servidor:

$$1000 \text{ pkt/s} \times 1024 \text{ bits/pkt} = 1.024.000 \text{ bps} \approx 1 \text{ Mbps}$$

5) Eficiencia de la línea:

La eficiencia es la relación entre lo que logramos transmitir (Caudal) y lo que la línea permite (Capacidad):

$$\text{Eficiencia} = \frac{1.024.000 \text{ bps}}{10^9 \text{ bps}} = 0.001024$$

Por lo tanto, la eficiencia de la línea durante la llamada RPC es apenas del **0.1024%**.

- 8) Considere de nuevo la situación del problema anterior. Calcule el tiempo de respuesta mínimo posible tanto para la línea de 1 Gbps dada como para una línea de 1 Mbps. ¿Qué conclusión puede sacar?

Para calcular el tiempo de propagación mínimo, debemos usar el tiempo de propagación (lo que tarda la señal en viajar) y el tiempo de transmisión (lo que tarda el equipo en poner los bits en línea).

Como en el ejercicio anterior, utilizaremos el retardo de propagación de $D = 0.5\text{ms}$ (y un $RTT = 1\text{ms}$) y el tamaño de paquete de **1024 bits**.

1) **Cálculo para la línea de 1 Mbps (10^6 bps):**

- Tiempo de transmisión: $\frac{1024\text{bits}}{10^6\text{bps}} = 0.001024\text{s} \approx 1.024\text{ms}$
- Tiempo de respuesta mínimo (transmisión + RTT): $1.024\text{ms} + 1\text{ms} = 2.024\text{ms}$

2) **Cálculo para la línea de 1 Gbps (10^9 bps):**

- Tiempo de transmisión: $\frac{1024\text{bits}}{10^9\text{bps}} = 0.000001024\text{s} \approx 0.01\text{ms}$
- Tiempo de respuesta mínimo (transmisión + RTT): $0.01\text{ms} + 1\text{ms} = 1.001\text{ms}$

La conclusión directa es que **el algoritmo de parada y espera "no es eficiente para redes con $BD \gg 1$ "** (redes con un producto de Ancho de Banda por Retardo muy grande).

A pesar de que multiplicamos la capacidad de la línea por 1000 (pasando de 1 Mbps a 1 Gbps), el tiempo de respuesta total **solo se redujo a la mitad** (de ~ 2 ms a ~ 1 ms). Esto demuestra que en enlaces de alta velocidad, el tiempo de transmisión se vuelve insignificante y el **retardo de propagación (RTT) domina por completo la comunicación**. Es por este motivo que protocolos como TCP utilizan ventanas deslizantes para serializar los datos y mantener la red ocupada en lugar de esperar una respuesta por cada paquete enviado.

- 9) Tanto UDP como TCP utilizan números de puerto para identificar la entidad de destino al entregar un mensaje. Dé dos razones por las que estos protocolos inventaron un nuevo identificador abstracto (número de puerto), en lugar de utilizar identificadores de proceso, que ya existían cuando se diseñaron estos protocolos.

- 1) **Funcionan como buzones independientes:** En la teoría los puertos se definen como "buzones locales que son cogidos prestados por los procesos". Esto implica que el puerto no está atado a la vida o identifiacior del proceso en sí, sino que es un punto de entrega abstracto en el sistema que cualquier proceso de la aplicación puede solicitar y usar.
- 2) **Permiten la estandarización global ("puertos bien conocidos"):** El texto indica que los servidores a menudo se asocian a "puertos bien conocidos" (por ejemplo, el puerto 80 para HTTP o el 22 para SSH). Si se dependiera de un identificador de proceso interno (el cual es asignado de forma dinámica y aleatoria por cada sistema operativo al arrancar un programa), sería imposible que los clientes de todo el mundo supieran de antemano a qué número conectarse para acceder a una página web o a un servidor de correo.

- 10) La fragmentación y el reesamblaje de datagramas son gestionados por IP y son invisibles para TCP. ¿Significa esto que TCP no tiene que preocuparse de que los datos lleguen en el orden equivocado?

La respuesta es **no, TCP sí tiene que preocuparse activamente por el orden de los datos**.

Aunque la capa inferior (IP) gestione sus propios procesos, una de las responsabilidades clave de TCP es garantizar un "flujo de datos fiable", asegurando que "la entrega es fiable y ordenada (los datos se entregan en el mismo orden que fueron enviados)".

Para lograr esto, TCP no confía ciegamente en que las capas inferiores le entreguen todo perfectamente ordenado, sino que implementa un mecanismo propio:

- TCP utiliza un diseño de **recepción selectiva** en su ventana deslizante.
- Si los segmentos de datos llegan desordenados, el receptor TCP no se los entrega directamente a la aplicación, sino que **"almacena los segmentos fuera de secuencia"** en un búfer.

- Una vez que tiene todas las piezas correctas, es el propio TCP quien **”le pasa los datos a la capa de aplicación en orden”**.

Por lo tanto, la gestión del orden de los segmentos es una tarea crítica y visible de la que se encarga la capa de transporte mediante TCP.

- 11) A un proceso en el host 1 se le ha asignado el puerto p , y a un proceso en el host 2 se le ha asignado el puerto q . ¿Es posible que haya dos o más conexiones TCP entre estos dos puertos al mismo tiempo?

La respuesta es **no**, no es posible tener dos o más conexiones TCP simultáneas entre los mismos puertos de esos dos hosts. Los siguientes dos fundamentos técnicos demuestran esta imposibilidad:

- 1) **Identificación única de los procesos:** ”un proceso de una aplicación se identifica por la tupla dirección IP, protocolo y puerto”. Si fijamos los hosts (las direcciones IP) y los puertos (p y q), no existe otro parámetro o identificador adicional que permita a la capa de transporte distinguir el tráfico de una hipotética segunda conexión simultánea entre esos mismos procesos.
- 2) **El estado TIME_WAIT (Prueba concluyente):** Este mecanismo obliga a esperar un tiempo largo (el doble del tiempo de vida de un segmento) antes de cerrar completamente una conexión terminada. La razón es evitar que mensajes retrasados o perdidos puedan **”interferir con una subsiguiente conexión”**. Esto demuestra que las conexiones entre los mismos identificadores no pueden coexistir en paralelo, sino que deben establecerse de forma secuencial, ya que el protocolo no tiene forma de separarlas si se solapan en el tiempo.

- 12) La carga útil máxima de un segmento TCP es de 65.495 bytes. ¿Por qué se eligió un número tan extraño?

Aunque a primera vista, 65.495 parece un número completamente aleatorio o un ”número mágico” sacado de la manga por los ingenieros, en realidad, no tiene nada de extraño. Es el resultado directo de una resta muy precisa basada en cómo están estructurados los protocolos de Internet.

- **El Límite de Paquete IP (65.535 bytes):** Todo segmento TCP viaja encapsulado dentro de un paquete IP (normalmente IPv4). En la cabecera de IPv4, el campo que define la ”Longitud Total” del paquete tiene un tamaño de exactamente 16 bits. El valor máximo que se puede representar con 16 bits es **65.535** ($2^{16} - 1$). Por lo tanto, un paquete IP completo jamás puede superar ese tamaño.
- **El ”Impuesto” de la Cabecera IP (20 bytes):** Para que ese paquete IP llegue a su destino, necesita llevar su propia información de control (direcciones IP de origen y su destino, tiempo de vida, etc.). El tamaño mínimo obligatorio de una cabecera IPv4 es de **20 bytes**.
- **El ”Impuesto” de la Cabecera TCP (20 bytes):** Dentro del paquete IP va el segmento TCP, que a su vez requiere su propia cabecera para gestionar la conexión (puertos, números de secuencia, ventanas deslizantes, etc.). El tamaño mínimo de una cabecera TCP también es de **20 bytes**.

La Matemática Final

Si tomamos el tamaño máximo absoluto que puede tener un paquete y le restamos el espacio que ocupan de forma obligatoria las cabeceras de control, obtenemos el espacio restante para tus datos (la carga útil o *payload*):

$$65.535 - 20 - 20 = 65.495 \text{ bytes}$$

- 13) Considera el efecto de utilizar el arranque lento en una línea con un tiempo de ida y vuelta de 10 ms y sin congestión. La ventana de recepción es 24 KB y el tamaño máximo del segmento es de 2 KB. ¿Cuánto tarda en enviarse la primera ventana completa?

Datos del problema

- Tiempo de ida y vuelta (RTT): 10 ms
- Ventana de recepción (*rwnd*): 24 KB

- Tamaño Máximo del Segmento (MSS): 2 KB/segmento
- Tamaño de la ventana en segmentos: $\frac{24 \text{ KB}}{2 \text{ KB/segmento}} = 12 \text{ segmentos}$.

Análisis del Comienzo Lento

La fase de comienzo lento se caracteriza porque **”se comienza por doblar el valor de cwnd cada RTT unidades de tiempo”**. Aplicando esta secuencia temporalmente con nuestro RTT de 10 ms:

- $t = 0$ ms: El valor inicial es de **1 segmento** (2 KB)
- $t = 10$ ms: Se dobla el valor a **2 segmentos** (4 KB)
- $t = 20$ ms: Se dobla el valor a **4 segmentos** (8 KB)
- $t = 30$ ms: Se dobla el valor a **8 segmentos** (16 KB)
- $t = 40$ ms: Se dobla el valor a **16 segmentos** (32 KB)

A los **40 ms (4 RTTs)**, el valor de la ventana de congestión alcanza los 16 segmentos, superando por primera vez la capacidad total de la ventana de recepción (12 segmentos). por lo tanto, se tarda **40 ms** en alcanzar la capacidad necesaria para poder enviar la primera ventana completa de 24 KB.

- 14)** Si el tiempo de ida y vuelta TCP, RTT, es actualmente de 30 mseg y los siguientes acuses de recibo llegan después de 26, 32 y 24 mseg, respectivamente, ¿cuál es la nueva estimación del RTT utilizando el algoritmo de Jacobson? Utilice ≤ 0.9 .

La estimación del RTT se realiza mediante el cálculo de las **”estimaciones suavizadas del RTT”** ($SRTT$), utilizando la siguiente fórmula:

$$SRTT_{N+1} = 0.9 \cdot SRTT_N + 0.1 \cdot RTT_{N+1}$$

Aplicamos esta fórmula paso a paso con los datos proporcionados, partiendo de una estimación inicial $SRTT_0 = 30$ ms.

Cálculo paso a paso:

- 1) Para el primer acuse de recibo ($RTT_1 = 26$ ms):

$$SRTT_1 = 0.9 \cdot SRTT_0 + 0.1 \cdot RTT_1 = 0.9 \cdot 30 + 0.1 \cdot 26 = 27 + 2.6 = 29.6 \text{ ms}$$

- 2) Para el segundo acuse de recibo ($RTT_2 = 32$ ms):

$$SRTT_2 = 0.9 \cdot SRTT_1 + 0.1 \cdot RTT_2 = 0.9 \cdot 29.6 + 0.1 \cdot 32 = 26.64 + 3.2 = 29.84 \text{ ms}$$

- 3) Para el tercer acuse de recibo ($RTT_3 = 24$ ms):

$$SRTT_3 = 0.9 \cdot SRTT_2 + 0.1 \cdot RTT_3 = 0.9 \cdot 29.84 + 0.1 \cdot 24 = 26.856 + 2.4 = 29.256 \text{ ms}$$

Tras procesar los tres acuses de recibo utilizando el algoritmo de Jacobson, la nueva estimación del RTT ($SRTT$) es de **29.256 ms**.

- 15)** Una máquina TCP está enviando ventanas completas de 65.535 bytes sobre un canal de 1 Gbps que tiene un retardo unidireccional de 10 mseg. ¿Cuál es el rendimiento máximo alcanzable? ¿Cuál es la eficiencia de la línea?

Datos iniciales

- Tamaño de ventana (W) : 65.535 bytes = $65.535 \times 8 = 524.280$ bits
- Capacidad del canal (R) : 1 Gbps = 10^9 bps.
- Retardo unidireccional (D): 10 ms = 0.01 s.

1) Cálculo del Tiempo de Ida y Vuelta (RTT)

$$RTT = 2 \times D = 2 \times 0.01s = 0.02s$$

2) Rendimiento Máximo Alcanzable (Caudal o *Throughput*)

La tasa de transmisión de TCP es aproximadamente el valor de la ventana dividido entre el RTT. De igual forma, el algoritmo de ventana deslizante permite enviar exactamente W datos por cada RTT.

$$\text{Rendimiento} = \frac{W}{RTT} = \frac{54,280 \text{ bits}}{0.02s} = 26,214,000 \text{ bps} = 26.214 \text{ Mbps}$$

3) Eficiencia de la Línea

La eficiencia es la relación entre el rendimiento real que estamos logrando inyectar en la red y la capacidad total bruta que ofrece el canal físico.

$$\text{Eficiencia} = \frac{\text{Rendimiento}}{\text{Capacidad}} = \frac{26,214,000 \text{ bps}}{10^9 \text{ bps}} = 0.026214$$

Podemos ver que la eficiencia de la línea es apenas del 2.62%.

16) ¿Cuál es la velocidad de línea más rápida a la que un host puede enviar cargas útiles TCP de 1.500 bytes con una duración máxima de 120 segundos sin que los números de secuencia se enrolen? Tenga en cuenta la sobrecarga TCP, IP y Ethernet. Suponga que las tramas Ethernet pueden enviarse continuamente.

1) Capacidad del número de secuencia.

Según el diagrama de la cabecera TCP, el campo "Número de secuencia" ocupa exactamente el ancho completo de la fila, es decir, **32 bits**. Esto significa que el contador de secuencias puede direccionar hasta 2^{32} bytes de datos antes de volver a cero (enrollarse).

2) Tasa máxima de carga útil (Payload)

Para asegurar que ningún número de secuencia se enrolle (se repita) mientras un paquete antiguo sigue vivo en la red (120 segundos de duración máxima), el host no puede agotar los 2^{32} bytes en menos de ese tiempo.

- **Tasa de datos útil** = $2^{32} \text{ bytes} / 120 \text{ segundos} = 35,791,394.13 \text{ bytes/s}$

3) Tasa de envío de paquetes

Si cada carga útil TCP enviada es de 1,500 bytes, el número de paquetes que se deben enviar por segundo para alcanzar esa tasa es:

- **Paquetes por segundo** = $35,791,394.13 / 1,500 \approx 23,860.93 \text{ paquetes/s}$

4) Tamaño total de la trama en la línea (Sobrecarga)

Aquí sumamos las sobrecargas (overhead). La cabecera TCP ocupa al menos 5 filas de 32 bits (4 bytes cada una) antes de las opciones, lo que da un mínimo de 20 bytes. Para el resto, usamos valores estándar:

- Carga útil TCP: 1500 bytes
- Cabecera TCP: 20 bytes
- Cabecera IP (estándar): 20 bytes
- Cabecera Ethernet y tráiler (estándar MAC + FCS): 18 bytes

$$\text{Tamaño total de la trama: } 1500 + 20 + 20 + 18 = 1558 \text{ bytes}$$

5) Velocidad de línea máxima

Multiplicamos los paquetes por segundo por el tamaño de la trama y luego por 8 para obtener los bits por segundos (bps):

- Velocidad en bytes/s = $13869.93 \text{ paquetes/s} \times 1558 \text{ bytes/paquetes} \approx 37175.329 \text{ bytes/s}$
- Velocidad de línea (bits/s) = $37175.329 \text{ bytes/s} \times 8 \text{ bits/byte} \approx 297402.632 \text{ bps}$

Por tanto, la velocidad de línea más rápida a la que el host puede enviar sin que los números de secuencia se enrollen antes de 120 segundos es de aproximadamente **297.4 Mbps**.

- 17) Para hacer frente a las limitaciones de la versión 4 de IP, se tuvo que realizar un gran esfuerzo a través del IETF que dio como resultado el diseño de la versión 6 de IP y todavía existen importantes reticencias en la adopción de esta nueva versión. Sin embargo, no es necesario un esfuerzo tan importante para abordar las limitaciones de TCP. Explique por qué es así.

La razón por la que abordar las limitaciones de TCP requiere un esfuerzo mucho menor que cambiar el protocolo IP radica en **dónde operan y quién los gestiona**:

- **TCP opera de extremo a extremo (Actualización local):** La capa de transporte, a la que pertenece TCP, proporciona "conectividad extremo a extremo a través de la red". Además, los mecanismos de TCP operan en un modelo "gestionado por el cliente". Esto significa que implementar una mejora, una nueva política de control de congestión o solucionar una limitación en TCP, por lo que general solo es necesario actualizar el software de los sistemas finales (es decir, el ordenador transmisor y el receptor).
- **IP opera en toda la red (Actualización global):** Por el contrario, la capa de red (IP) es la infraestructura subyacente distribuida. El problema al intentar añadir una mejora que dependa de la capa de red, como la *Notificación explícita de la congestión*: señala como desventaja principal que **"Los encaminadores y los equipos deben actualizarse"**.

En resumen, modificar TCP es un esfuerzo menor porque solo dependes de actualizar los dispositivos de los usuarios en los extremos de la comunicación. Modificar la capa de red (como hacer la transición a IPv6) es un esfuerzo titánico porque requiere actualizar físicamente o reconfigurar los encaminadores (routers) y conmutadores que forman el núcleo de todo Internet.

- 18) En una red cuyo segmento máximo es de 128 bytes, la duración máxima del segmento es de 30 segundos y tiene números de secuencia de 8 bits, ¿cuál es la velocidad máxima de datos por conexión?

Podemos analizar cómo funcionan los números de secuencia y el riesgo de que se repitan (se enrollen) mientras un paquete antiguo sigue existiendo en la red.

1) Capacidad del número de secuencia

Los números de secuencia se implementan típicamente con un contador de N bits que vuelve a 0 después de alcanzar el valor $2^N - 1$. Si utilizamos un número de secuencia de 8 bits ($N = 8$), disponemos de **256 números de secuencia únicos** (del 0 al 255) para identificar los paquetes (segmentos).

2) Tasa máxima de paquetes para evitar el enrollamiento

Para evitar que la red confunda un paquete nuevo con una copia de un paquete antiguo que se haya quedado rezagado, los números de secuencia no pueden dar una vuelta completa (agotarse y volver a empezar) en un tiempo inferior a la **duración máxima del segmento en la red (30 segundos)**.

Por lo tanto, el límite de seguridad es enviar un máximo de 256 paquetes cada 30 segundos:

$$\text{Tasa de paquetes} = \frac{256 \text{ paquetes}}{30\text{s}} \approx 8.533 \text{ paquetes/s}$$

3) Velocidad máxima de datos

Sabiendo cuántos paquetes podemos enviar por segundo, solo tenemos que multiplicar esa cantidad por el máximo de cada segmento (128 bytes) para obtener la velocidad de datos:

$$\text{Velocidad en bytes} = 8.533 \text{ paquetes/s} \times 128 \text{ bytes/paquete} = 1092.26 \text{ bytes/s} \equiv 8738.13\text{bps}$$

Por lo tanto, la velocidad máxima de datos por conexión en estas condiciones es de aproximadamente **8.74 kbps**.

- 19) Suponga que está midiendo el tiempo de recepción de un segmento. Cuando se produce una interrupción, usted lee el reloj del sistema en milisegundos. Cuando el segmento se ha procesado por completo, se vuelve a leer el reloj. Mides 0 mseg 270.000 veces y 1 mseg 730.000 veces. ¿Cuánto tarda en recibirse un segmento?
- 20) Para evitar el problema de que los números de secuencia se desvíen mientras aún existen paquetes antiguos, se podrían utilizar números de secuencia de 64 bits. Sin embargo, teóricamente, una fibra óptica puede funcionar a 75 Tbps. ¿Qué duración máxima de los paquetes es necesaria para asegurar que las futuras redes de 75 Tbps no tengan problemas de wraparound incluso con números de secuencia de 64 bits? Suponga que cada byte tiene su propio número de secuencia, como hace TCP.

- 1) Capacidad total de los números de secuencia

Si utilizamos un contador de 64 bits ($N = 64$), la cantidad total de números de secuencia únicos disponibles (y por tanto, de bytes que podemos enumerar) es 2^{64} bytes.

- 2) Tasa de transmisión en bytes

La capacidad de la línea es de 75 Tbps. Para saber a qué velocidad agotamos los números de secuencia (que cuentan bytes), debemos convertir esta velocidad de bits a bytes:

$$\text{Velocidad} = \frac{75 \cdot 10^{12} \text{ bits/s}}{8 \text{ bits/bytes}} = 9.375 \cdot 10^{12} \text{ bytes/s}$$

- 3) Tiempo máximo antes del enrollamiento

Para evitar problemas, la duración máxima de un paquete en la red no debe superar el tiempo que tarda la conexión en utilizar todos los números de secuencia y volver a empezar. Dividimos el total de bytes direccionables entre la velocidad de transmisión:

$$T = \frac{2^{64} \text{ bytes}}{9.375 \cdot 10^{12} \text{ bytes/s}} \approx 1967.652 \text{ s}$$

Si convertimos estos segundos a unidades más legibles:

- En horas: ~ 546.5 horas
- En días: ~ 22.7 días

- 21) Para una red de 1 Gbps que opera a más de 4000 km, el retardo es el factor limitante, no el ancho de banda. Considere un MAN con una distancia media entre el origen y el destino de 20 km. ¿A qué velocidad de datos el retardo de ida y vuelta debido a la velocidad de la luz es igual al retardo de transmisión de un paquete de 1 KB?

- 1) Cálculo del Retardo de Ida y Vuelta (RTT)

El RTT equivale al doble del retardo de propagación unidireccional ($2D$).

Calculamos primero el retardo unidireccional (D) para una distancia de 20 km:

$$D = \frac{20 \text{ km}}{2 \times 10^5 \text{ km/s}} = 0.0001 \text{ s} = 0.1 \text{ ms} \implies RTT = 2 \times 0.0001 \text{ s} = 0.0002 \text{ s}$$

- 2) Definición del Tiempo de Transmisión

El tiempo de transmisión es lo que tarda el equipo en inyectar todos los bits del paquete en la línea. Se calcula dividiendo el tamaño del paquete (L) entre la velocidad de datos (R).

- **Tamaño del paquete (L):** 1 KB = 1024 bytes = 8192 bits

3) Igualación y cálculo de la velocidad (R)

El problema nos pide que el retardo de ida y vuelta (RTT) sea igual al tiempo de transmisión:

$$RTT = \frac{L}{R}$$

Despejamos la velocidad de datos (R):

$$R = \frac{L}{RTT} = \frac{8192\text{bits}}{0.0002\text{s}} = 40960000\text{bps}$$

Para que el retardo de ida y vuelta sea exactamente igual al retardo de transmisión de un paquete de 1 KB a una distancia de 20 km, la velocidad de datos de la red debe ser de **40.96 Mbps**.

22) Calcule el producto ancho de banda-retardo para las siguientes redes: (1) T1 (1,5 Mbps), (2) Ethernet (10 Mbps), (3) T3 (45 Mbps) y (4) STS-3 (155 Mbps). Supongamos un RTT de 100 ms. Recuerda que una cabecera TCP tiene 16 bits reservados para el tamaño de ventana. ¿Cuáles son sus implicaciones a la luz de tus cálculos?

1) Cálculo del Producto Ancho de Banda-Retardo

1) Red T1 (1.5 Mbps):

$$BDP = \frac{1.5 \cdot 10^6\text{bps} \times 0.1\text{s}}{8 \text{ bits/byte}} = 18750 \text{ bytes}$$

2) Red Ethernet (10 Mbps):

$$BDP = \frac{10 \cdot 10^6\text{bps} \times 0.1\text{s}}{8 \text{ bits/byte}} = 125000 \text{ bytes}$$

3) Red T3 (45 Mbps):

$$BDP = \frac{45 \cdot 10^6\text{bps} \times 0.1\text{s}}{8\text{bits/byte}} = 562500\text{bytes}$$

4) Red STS-3 (155 Mbps):

$$BDP = \frac{155 \cdot 10^6\text{bps} \times 0.1\text{s}}{8 \text{ bits/byte}} = 1937500 \text{ bytes}$$

2) Implicaciones sobre el tamaño de la ventana TCP

Los diagramas de la cabecera TCP muestran que el campo "**Tamaño de venta**" ocupa exactamente **16 bits**.

Con 16 bits, el valor máximo matemático que un receptor puede anunciar es de $2^{16} - 1 = 65535$ bytes. La ventana deslizante "utiliza la serialización para mantener a la red ocupada" y que el transmisor puede enviar un máximo de W datos hasta que se asienten.

¿Qué pasa a la luz de nuestros cálculos?

- En la **red T1**, el BDP es menor que el máximo de la ventana TCP. Esto significa que la ventana estándar de 16 bits es suficiente para llenar la tubería y lograr el 100% de eficiencia.
- En las **redes Ethernet, T3 y STS-3**, el BDP supera con creces el límite de 65535 bytes.

La implicación directa:

En los enlaces de 10 Mbps, 45 Mbps y 155 Mbps con ese nivel de retardo, el campo de 16 bits de la cabecera TCP clásica se convierte en un cuello de botella restrictivo. El transmisor alcanzará el límite de la ventana y se verá obligado a detenerse y esperar a recibir asentimientos (ACKs) mucho antes de haber llenado la capacidad de la línea. Como resultado, **será imposible aprovechar el ancho de banda total** de estas redes de alta velocidad, ya que la red pasará una fracción importante del tiempo inactiva esperando las confirmaciones.

23) ¿Cuál es el producto ancho de banda-retardo para un canal de 50 Mbps en un satélite geoestacionario? Si los paquetes son todos de 1500 bytes (incluyendo la sobrecarga), ¿qué tamaño debe tener la ventana en paquetes?

Como en el enunciado no se especifica el tiempo de ida y vuelta (RTT), asumiremos el estándar de aproximadamente 0.5 s para realizar los cálculos.

1) Cálculo del Producto Ancho de Banda-Retardo (BDP)

Para aprovechar la serialización y "rellenar el camino de red", es necesario calcular la cantidad de datos en tránsito, lo que se define como $2BD$ (donde B es el ancho de banda y $2D$ es el RTT).

- Ancho de banda (B) : 50Mbps = $50 \cdot 10^6$ bps.
- Retardo (RTT o $2D$): 0.5 s

Multiplicamos ambos valores para obtener el BDP en bits:

$$BDP = 50 \cdot 10^6 \text{bps} \times 0.5\text{s} = 25 \cdot 10^6 \text{bits} = 3.125 \cdot 10^6 \text{bytes}$$

2) Tamaño de la ventana en paquetes

El algoritmo de la venta deslizante requiere que el transmisor tenga un tamaño de ventana (W) lo suficientemente grande como para abarcar todo este producto y así "mantener la red ocupada" sin detenerse a esperar asentimientos.

Si todos los paquetes tienen un tamaño exacto de 1500 bytes (incluyendo la sobrecarga), dividimos el BDP total entre el tamaño del paquete:

$$W(\text{en paquetes}) = \frac{3.125 \cdot 10^6 \text{ bytes}}{1500 \text{ bytes/paquete}} = 2083.33 \text{ paquetes}$$

Dado que no podemos enviar fracciones de un paquete, debemos redondear hacia arriba para garantizar que la tubería se llene por completo, por lo tanto, el tamaño que debe tener la venta es de **2084 paquetes**.